

1 Native order and multiplicative annuli

Source: PrimeTensor/Order.lean

What this file establishes

Two earlier modules left a gap that this one closes. `PrimeTensor/Ratio.lean` defined the cross-product strict order `Lt` on representatives and proved nothing about it; `PrimeTensor/MulRat.lean` took the quotient and transported only the multiplicative structure across, leaving `MulRat` with no order at all. Here `Lt` is shown to respect the equivalence, lifted to `MulRat`, and proved irreflexive and transitive.

On top of that comes the notion the completion will actually be built from: $\text{Within}(\delta, a, b)$, saying that a and b lie in a common multiplicative δ -annulus. There is no distance function anywhere, and no subtraction — closeness is expressed by bounding a ratio in both directions, and the tolerance is a magnitude exceeding 1 rather than a small positive number.

1.1 Positivity, again

Lemma 1.1 (`PrimeRatio.multiset_eval_pos_order`). $0 < \text{eval}(q)$ for every $q : \text{PrimeMultiset}$.

Proof. By `Quotient.inductionOn`, reducing to a representative bag, where $0 < 1$ and `PrimeBag.one_le_eval` compose. $\square$

This is word for word the lemma proved in `PrimeTensor/MulRat.lean`, restated because that one is `private` and so not visible outside its own file. Both are `private`, and the names differ only by the suffix `_order`. Everything below rests on it: each order proof cancels a factor, and cancellation of a strict inequality on $\mathbb{N}$ is valid only against a positive factor.

1.2 The order respects the equivalence

Lemma 1.2 (Left congruence, `PrimeRatio.lt_same_left`). If $\text{Same}(a, a')$ then $\text{Lt}(a, b) \leftrightarrow \text{Lt}(a', b)$.

```
private theorem lt_of_same_left {a a' b : PrimeRatio}
  (ha : Same a a') (h : Lt a b) : Lt a' b := by
  unfold Same at ha
  unfold Lt at h ⊢
  exact (Nat.mul_lt_mul_right (multiset_eval_pos_order a.lower)).mp (by
    calc
      (a'.upper.eval * b.lower.eval) * a.lower.eval
        = (a'.upper.eval * a.lower.eval) * b.lower.eval := by ac_rfl
      _ = (a.upper.eval * a'.lower.eval) * b.lower.eval := by rw [← ha]
      _ = (a.upper.eval * b.lower.eval) * a'.lower.eval := by ac_rfl
      _ < (b.upper.eval * a.lower.eval) * a'.lower.eval :=
        (Nat.mul_lt_mul_right (multiset_eval_pos_order a'.lower)).mpr h
      _ = (b.upper.eval * a'.lower.eval) * a.lower.eval := by ac_rfl)
```

Proof. Write $u_x = \text{eval}(x.\text{upper})$, $l_x = \text{eval}(x.\text{lower})$. The forward direction is the private lemma `lt_of_same_left` displayed above. Its hypotheses are $u_a l_{a'} = u_{a'} l_a$ and $u_a l_b < u_b l_a$, and its goal is $u_{a'} l_b < u_b l_{a'}$.

Since $\mathbb{N}$ has no division, the goal is multiplied by l_a and the factor cancelled at the end: by `Nat.mul_lt_mul_right` against the positive l_a it suffices to show $(u_{a'}l_b)l_a < (u_b l_{a'})l_a$, which is the displayed chain — reassociate to expose $u_{a'}l_a$, rewrite it backwards along the hypothesis to $u_a l_{a'}$, reassociate to expose $u_a l_b$, apply the strict hypothesis multiplied by the positive $l_{a'}$, and reassociate once more. Every equality step is `ac_rfl`.

The stated `iff` then follows by applying the private lemma in both directions, the second time to `same_symm ha`. $\square$

Lemma 1.3 (Right congruence, `PrimeRatio.lt_same_right`). *If $\text{Same}(b, b')$ then $\text{Lt}(a, b) \leftrightarrow \text{Lt}(a, b')$.*

Proof. The mirror image of Lemma 1.2, via the private `lt_of_same_right`: multiply by l_b , and this time apply the strict hypothesis *first* and rewrite along `Same(b, b')` afterwards, since the hypothesis being replaced now sits on the right of the inequality. $\square$

Lemma 1.4 (`PrimeRatio.lt_iff_of_same`). *If $\text{Same}(a, a')$ and $\text{Same}(b, b')$ then $\text{Lt}(a, b) \leftrightarrow \text{Lt}(a', b')$.*

Proof. Compose Lemmas 1.2 and 1.3: replacing one argument at a time is enough, and the two replacements are independent. $\square$

1.3 Irreflexivity and transitivity at representative level

Lemma 1.5 (`lt_irrefl_ratio`). $\neg \text{Lt}(a, a)$ for every $a : \text{PrimeRatio}$.

Proof. Unfolded, $\text{Lt}(a, a)$ is $u_a l_a < u_a l_a$, contradicted by irreflexivity of $<$ on $\mathbb{N}$. Note that no positivity is needed here — the two sides are literally the same natural number. $\square$

Lemma 1.6 (`lt_trans_ratio`). *If $\text{Lt}(a, b)$ and $\text{Lt}(b, c)$ then $\text{Lt}(a, c)$.*

Proof. The hypotheses are $u_a l_b < u_b l_a$ and $u_b l_c < u_c l_b$ and the goal is $u_a l_c < u_c l_a$. Multiplying the goal by the positive l_b and cancelling afterwards, it suffices to prove $(u_a l_c)l_b < (u_c l_a)l_b$, and the chain runs

$$(u_a l_c)l_b = (u_a l_b)l_c < (u_b l_a)l_c = (u_b l_c)l_a < (u_c l_b)l_a = (u_c l_a)l_b,$$

with the two strict steps being the hypotheses multiplied by the positive l_c and l_a respectively, and the three equalities `ac_rfl`. $\square$

Remark 1.7 (One pattern, three times). Lemmas 1.2, 1.3 and 1.6 are the same manoeuvre: multiply through by the denominator that the conclusion does not mention, walk a chain of `ac_rfl` reassociations punctuated by the hypotheses, then cancel. It is the strict-inequality counterpart of the calculation proving transitivity of `Same` in `PrimeTensor/MulRat.lean`, and it exists for the same reason: the natural formulation would divide, and $\mathbb{N}$ has no division. Each use needs a positive factor, and each gets it from Lemma 1.1.

1.4 Lifting the order to the quotient

Definition 1.8 (`MulRat.lt` and the `LT` instance). `lt : MulRat → MulRat → Prop` is `PrimeRatio.Lt` lifted through the quotient in both arguments, and `instance : LT MulRat` makes $a < b$ mean `lt(a, b)`.

```
def lt (a b : MulRat) : Prop :=
  Quotient.liftOn₂ a b
    PrimeRatio.Lt
  (by
    intro a₁ b₁ a₂ b₂ ha hb
    change PrimeRatio.Same a₁ a₂ at ha
    change PrimeRatio.Same b₁ b₂ at hb
    apply propext
    exact PrimeRatio.lt_iff_of_same ha hb)
```

Remark 1.9 (This lift needs `propext`). The lifts in `PrimeTensor/MulRat.lean` produced elements of `MulRat`, and their obligations were discharged by `Quotient.sound`, which turns a `Same` into an equality of classes. This one produces an element of `Prop`, so its obligation is that the two propositions are *equal*, not merely equivalent. Lemma 1.4 supplies an $\leftrightarrow$, and `propext` — propositional extensionality — converts it to the required equality. That axiom is the only extra principle the order costs, and it is one Lean’s standard library assumes throughout.

Theorem 1.10 (The lifted order is a strict order). *For $a, b, c : \text{MulRat}$: $\neg a < a$, and $a < b$ together with $b < c$ gives $a < c$. These are `MulRat.lt_irrefl` and `MulRat.lt_trans`.*

Proof. Both by `Quotient.inductionOn`, reducing to representatives, where the goals become $\neg \text{Lt}(x, x)$ and $\text{Lt}(x, z)$ and the representative-level Lemmas 1.5 and 1.6 apply. Transitivity inducts on all three arguments in turn, carrying the hypotheses along. No appeal to `Quotient.sound` is needed: the statements are propositions about classes, not equations between them. $\square$

Remark 1.11 (What is not established). No order-theoretic instance is registered beyond `LT`: there is no `Preorder`, `PartialOrder` or `LinearOrder` for `MulRat`, so `Mathlib`’s order lemmas do not apply and every fact must be cited from this file. Asymmetry is not stated, though it follows from irreflexivity and transitivity. Neither is trichotomy — that of $a < b$, $a = b$, $b < a$ exactly one holds — nor any compatibility between the order and the group structure, such as $a < b$ implying $ac < bc$. Nothing relates $<$ to `inv`.

1.5 Ratios and annuli

Definition 1.12 (`MulRat.ratio`). $\text{ratio}(a, b) \equiv a \cdot b^{-1}$.

Lemma 1.13 (`MulRat.ratio_self`). $\text{ratio}(a, a) = 1$.

Proof. Unfolding, this is $a \cdot a^{-1} = 1$, which is `MulRat.mul_inv` from `PrimeTensor/MulRat.lean`. It is tagged `@[simp]`. $\square$

This is a real strengthening over the representative level. In `PrimeTensor/Ratio.lean`, `PrimeRatio.ratio(a, a)` is the term $\langle u_a l_a, l_a u_a \rangle$, which is only `Same` to the unit, not equal to it. After the quotient the two coincide, so the self-ratio can be rewritten away by `simp` rather than carried as a relation.

Definition 1.14 (Annulus membership, `MulRat.Within`). For $\delta, a, b : \text{MulRat}$,

$$\text{Within}(\delta, a, b) \equiv (\text{ratio}(a, b) < \delta) \wedge (\text{ratio}(b, a) < \delta).$$

```
def Within (δ a b : MulRat) : Prop :=
  ratio a b < δ ∧ ratio b a < δ
```

Lemma 1.15 (`within_refl` and `within_symm`). *If $1 < \delta$ then $\text{Within}(\delta, a, a)$; and $\text{Within}(\delta, a, b)$ implies $\text{Within}(\delta, b, a)$.*

Proof. For reflexivity, Lemma 1.13 rewrites both conjuncts to $1 < \delta$, which is the hypothesis, supplied twice. Symmetry is the exchange of the two conjuncts, the definition being symmetric under swapping a and b . $\square$

Design note 1.16 (Why the tolerance exceeds one). The hypothesis $1 < \delta$ in `within_refl` is not a convenience; it is forced. Reflexivity requires $\text{ratio}(a, a) < \delta$, and $\text{ratio}(a, a)$ is exactly 1, so $\text{Within}(\delta, a, a)$ holds if and only if $1 < \delta$. More generally, $\text{ratio}(a, b)$ and $\text{ratio}(b, a)$ are mutually inverse, so at least one is ≥ 1 and a tolerance $\delta \leq 1$ admits no pair at all.

So $\delta > 1$ is the multiplicative reading of $\varepsilon > 0$, and $\delta = 1$ is the degenerate tolerance rather than the exact one — the opposite of what the additive analogy suggests. The module header says as much. Only the sufficient direction is a declaration; that $1 < \delta$ is also necessary is not stated in the file.

Remark 1.17 (No triangle inequality). `Within` is reflexive for admissible tolerances and symmetric, and that is all. The remaining ingredient of a metric-like notion — the multiplicative triangle inequality, that $\text{Within}(\delta, a, b)$ and $\text{Within}(\varepsilon, b, c)$ give $\text{Within}(\delta\varepsilon, a, c)$ — is not proved here, and neither is any statement combining two annuli. Since Cauchy convergence is what the header advertises this notion is for, the composition rule is the thing to look for in the modules that follow.

Summary of the correspondence

Lean declaration	Kind	Here
<code>PrimeRatio.multiset_eval_pos_order</code>	private thm	Lem. 1.1
<code>PrimeRatio.lt_of_same_left</code>	private thm	Lem. 1.2
<code>PrimeRatio.lt_same_left</code>	theorem	Lem. 1.2
<code>PrimeRatio.lt_of_same_right</code>	private thm	Lem. 1.3
<code>PrimeRatio.lt_same_right</code>	theorem	Lem. 1.3
<code>PrimeRatio.lt_iff_of_same</code>	theorem	Lem. 1.4
<code>PrimeRatio.lt_irrefl_ratio</code>	theorem	Lem. 1.5
<code>PrimeRatio.lt_trans_ratio</code>	theorem	Lem. 1.6
<code>MulRat.lt</code>	definition	Def. 1.8
<code>LT MulRat</code>	instance	Def. 1.8
<code>MulRat.lt_irrefl</code>	theorem	Thm. 1.10
<code>MulRat.lt_trans</code>	theorem	Thm. 1.10
<code>MulRat.ratio</code>	definition	Def. 1.12
<code>MulRat.ratio_self</code>	simp thm	Lem. 1.13
<code>MulRat.Within</code>	definition	Def. 1.14
<code>MulRat.within_refl</code>	theorem	Lem. 1.15
<code>MulRat.within_symm</code>	theorem	Lem. 1.15

Every declaration of `PrimeTensor/Order.lean` appears above. The file contains no `sorry`, no axiom of its own, and no unproved named proposition; it does use `propext`, as Remark 1.9 explains. The necessity half of Design note 1.16 is stated for the reader and is not a declaration of the file.